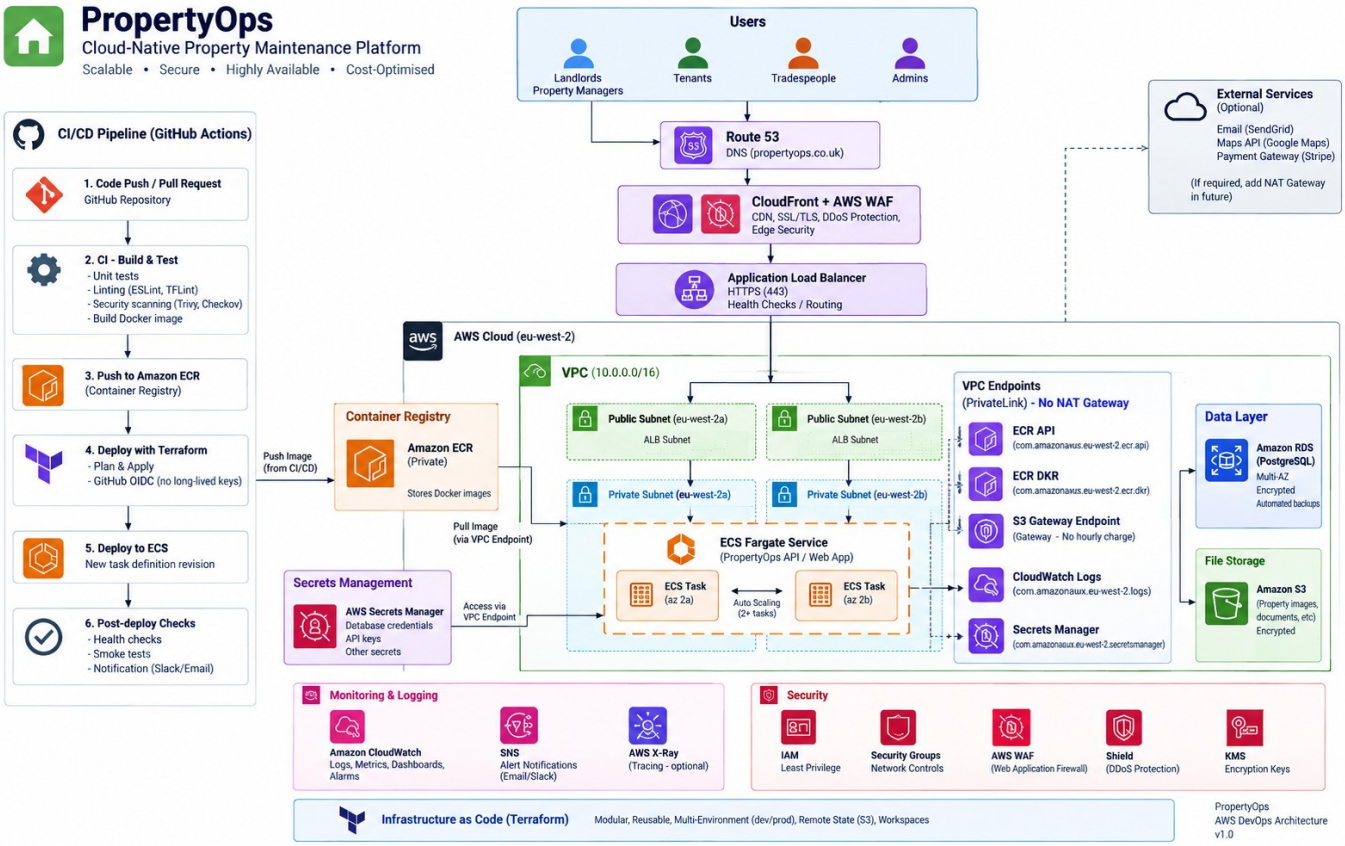
PropertyOps

ClickOps Build Guide

AWS ECS Fargate • Private Networking • VPC Endpoints • RDS • ECR • CloudWatch



Target architecture for the ClickOps phase (no NAT Gateway).

Goal: Build the complete PropertyOps platform manually in the AWS Console first, verify each layer, capture evidence, then recreate the same architecture with Terraform and CI/CD.

Region used throughout: eu-west-2 (London)

1. Build plan and naming convention

This guide deliberately builds the platform in dependency order. Do not jump ahead: networking and security groups must exist before ECS, RDS and the load balancer can be connected cleanly.

Component	Recommended name	Purpose
VPC	propertyops-vpc	Network boundary
ALB SG	propertyops-alb-sg	Public entry point
ECS SG	propertyops-ecs-sg	App traffic only from ALB
RDS SG	propertyops-rds-sg	PostgreSQL only from ECS
Endpoint SG	propertyops-vpce-sg	HTTPS from ECS to interface endpoints
ECR	propertyops-app	Private container repository
ECS Cluster	propertyops-cluster	Fargate cluster
ECS Service	propertyops-service	Runs app tasks
RDS	propertyops-db	PostgreSQL database
S3	propertyops-uploads-<unique>	Issue photos/documents

Cost control: Create an AWS Budget/alert before starting. Interface VPC endpoints, ALB, Fargate and RDS can incur hourly charges. Destroy lab resources after screenshots/testing.

Target subnet plan

Type	AZ	CIDR	Use
Public	eu-west-2a	10.0.0.0/24	ALB
Public	eu-west-2b	10.0.1.0/24	ALB
Private App	eu-west-2a	10.0.10.0/24	ECS tasks + interface endpoints
Private App	eu-west-2b	10.0.11.0/24	ECS tasks + interface endpoints
Private DB	eu-west-2a	10.0.20.0/24	RDS subnet group
Private DB	eu-west-2b	10.0.21.0/24	RDS subnet group

2. Create the VPC and six subnets

2.1 Create the VPC

Console path: VPC → Your VPCs → Create VPC

1. Choose VPC only.
2. Name tag: propertyops-vpc.
3. IPv4 CIDR: 10.0.0.0/16.
4. IPv6: none for this first build.
5. Create the VPC.
6. Open the VPC → Actions → Edit VPC settings → make sure DNS resolution and DNS hostnames are enabled.

Checkpoint: You should have one VPC with CIDR 10.0.0.0/16 and both DNS options enabled.

2.2 Create the six subnets

Console path: VPC → Subnets → Create subnet

7. Select propertyops-vpc.
8. Create the two public subnets using 10.0.0.0/24 in eu-west-2a and 10.0.1.0/24 in eu-west-2b.
9. Create the two private app subnets using 10.0.10.0/24 in eu-west-2a and 10.0.11.0/24 in eu-west-2b.
10. Create the two private DB subnets using 10.0.20.0/24 in eu-west-2a and 10.0.21.0/24 in eu-west-2b.
11. Use clear Name tags: propertyops-public-2a, propertyops-public-2b, propertyops-app-2a, propertyops-app-2b, propertyops-db-2a, propertyops-db-2b.

Checkpoint: VPC → Subnets should show six subnets spanning two Availability Zones.

2.3 Internet Gateway and public routing

Console path: VPC → Internet gateways / Route tables

12. Create an Internet Gateway named propertyops-igw and attach it to propertyops-vpc.
13. Create a route table named propertyops-public-rt.
14. Add route 0.0.0.0/0 → propertyops-igw.
15. Associate only the two public subnets with propertyops-public-rt.
16. Leave the private app and private DB subnets without an internet default route.

Checkpoint: The public subnets can route to the internet; private app/DB subnets remain private.

Cost / security note: Do not create a NAT Gateway. This design uses VPC endpoints for AWS-service access.

3. Create security groups

3.1 ALB security group

Console path: EC2 → Security Groups → Create security group

17. Name: propertyops-alb-sg; VPC: propertyops-vpc.
18. For the first smoke test, allow inbound TCP 80 from 0.0.0.0/0.
19. When HTTPS is configured later, add TCP 443 and remove/redirect HTTP as appropriate.
20. Leave outbound traffic allowed.

Checkpoint: The ALB SG is the only internet-facing security group.

3.2 ECS task security group

Console path: EC2 → Security Groups → Create security group

21. Name: propertyops-ecs-sg; VPC: propertyops-vpc.
22. Inbound: application port (use 80 for the starter container) with source = propertyops-alb-sg.
23. Do not allow 0.0.0.0/0 inbound to ECS tasks.
24. Outbound: allow all initially; tighten later if required.

Checkpoint: ECS tasks can only receive application traffic from the ALB.

3.3 RDS security group

Console path: EC2 → Security Groups → Create security group

25. Name: propertyops-rds-sg; VPC: propertyops-vpc.
26. Inbound: PostgreSQL TCP 5432, source = propertyops-ecs-sg.
27. No public inbound rules.

Checkpoint: Only ECS tasks can initiate PostgreSQL connections.

3.4 VPC endpoint security group

Console path: EC2 → Security Groups → Create security group

28. Name: propertyops-vpce-sg; VPC: propertyops-vpc.
29. Inbound: HTTPS TCP 443, source = propertyops-ecs-sg.
30. Outbound: allow all.

Checkpoint: The interface endpoints accept HTTPS from ECS tasks only.

4. Create a private ECR repository and starter container

4.1 Create ECR

Console path: Amazon ECR → Private registry → Repositories → Create repository

31. Repository name: propertyops-app.
32. Keep repository private.
33. Enable scan on push if shown in your console.
34. Enable encryption (default AWS-managed encryption is acceptable for the first build).
35. Create repository.

Checkpoint: Copy the repository URI; you will use it when tagging the Docker image.

4.2 Build a tiny starter app locally

Use a simple container first to prove the infrastructure. Replace it with the real PropertyOps application later.

```
mkdir propertyops-app && cd propertyops-app
cat > server.js <<'EOF'
const http = require('http');
const port = process.env.PORT || 80;
http.createServer((req,res) => {
  if (req.url === '/health') {
    res.writeHead(200, {'Content-Type':'application/json'});
    return res.end(JSON.stringify({status:'ok', service:'propertyops'}));
  }
  res.writeHead(200, {'Content-Type':'text/html'});
  res.end('<h1>PropertyOps</h1><p>Cloud-native property maintenance platform</p>');
}).listen(port);
EOF

cat > Dockerfile <<'EOF'
FROM node:22-alpine
WORKDIR /app
COPY server.js .
EXPOSE 80
CMD ["node","server.js"]
EOF

docker build -t propertyops-app .
```

4.3 Authenticate and push to ECR

```
aws ecr get-login-password --region eu-west-2 | \
docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.eu-west-2.amazonaws.com

docker tag propertyops-app:latest \
<ACCOUNT_ID>.dkr.ecr.eu-west-2.amazonaws.com/propertyops-app:latest

docker push \
<ACCOUNT_ID>.dkr.ecr.eu-west-2.amazonaws.com/propertyops-app:latest
```

Checkpoint: ECR → propertyops-app should show an image tagged latest.

5. Add VPC endpoints (no NAT Gateway)

Fargate tasks in private subnets need private connectivity to AWS services. For ECR image pulls on current Fargate platforms, use ECR API + ECR DKR interface endpoints and an S3 gateway endpoint; add CloudWatch Logs and Secrets Manager endpoints for logging and secrets.

Endpoint	Type	Subnets / route tables	Private DNS
com.amazonaws.eu-west-2.ecr.api	Interface	private app 2a + 2b	Enabled
com.amazonaws.eu-west-2.ecr.dkr	Interface	private app 2a + 2b	Enabled
com.amazonaws.eu-west-2.logs	Interface	private app 2a + 2b	Enabled
com.amazonaws.eu-west-2.secretsmanager	Interface	private app 2a + 2b	Enabled
com.amazonaws.eu-west-2.s3	Gateway	private app route table(s)	N/A

5.1 Create interface endpoints

Console path: VPC → Endpoints → Create endpoint

36. Service category: AWS services.
37. Create ecr.api, ecr.dkr, logs and secretsmanager endpoints for eu-west-2.
38. VPC: propertyops-vpc.
39. Subnets: select propertyops-app-2a and propertyops-app-2b.
40. Security group: propertyops-vpce-sg.
41. Enable Private DNS for each interface endpoint.
42. Keep the default endpoint policy initially.

Checkpoint: Four interface endpoints should show Available.

5.2 Create the S3 gateway endpoint

Console path: VPC → Endpoints → Create endpoint

43. Select service com.amazonaws.eu-west-2.s3.
44. Endpoint type: Gateway.
45. VPC: propertyops-vpc.
46. Associate it with the route table(s) used by the private app subnets.
47. Keep the default full-access endpoint policy initially.

Checkpoint: Private app routes should include the S3 prefix-list route through the gateway endpoint.

Cost / security note: S3 gateway endpoints have no additional endpoint hourly charge; interface endpoints do have hourly/per-GB charges.

6. Create PostgreSQL in private DB subnets

6.1 Create a DB subnet group

Console path: RDS → Subnet groups → Create DB subnet group

48. Name: propertyops-db-subnet-group.
49. VPC: propertyops-vpc.
50. Add only propertyops-db-2a and propertyops-db-2b.

Checkpoint: The DB subnet group spans two Availability Zones.

6.2 Create the database

Console path: RDS → Databases → Create database

51. Engine: PostgreSQL.
52. Template: choose a development/free-tier option if available for your account.
53. DB identifier: propertyops-db.
54. Use the smallest suitable instance class for the lab.
55. Storage: small gp3 allocation; enable storage encryption.
56. Connectivity: propertyops-vpc.
57. DB subnet group: propertyops-db-subnet-group.
58. Public access: No.
59. Security group: propertyops-rds-sg.
60. Database port: 5432.
61. Enable automated backups. Use Single-AZ during development if cost is a priority; Multi-AZ is the production-ready upgrade.

Checkpoint: Wait until RDS status is Available, then copy the endpoint hostname.

Cost / security note: RDS is one of the main ongoing-cost items. Multi-AZ increases resilience but also increases cost.

6.3 Store DB connection details in Secrets Manager

Create application database secret

Console path: Secrets Manager → Store a new secret

62. Secret type: Other type of secret or RDS credentials.
63. Store host, port, database, username and password.
64. Secret name: propertyops/database.
65. Keep encryption with the default key for the first build.

Checkpoint: The secret exists and can be selected from an ECS task definition.

7. Create the target group and Application Load Balancer

7.1 Target group

Console path: EC2 → Target Groups → Create target group

66. Target type: IP addresses.
67. Name: propertyops-tg.
68. Protocol: HTTP; Port: 80.
69. VPC: propertyops-vpc.
70. Health check path: /health.
71. Do not manually register targets; ECS will register task IPs.

Checkpoint: Target group exists with zero targets before the ECS service is created.

7.2 Application Load Balancer

Console path: EC2 → Load Balancers → Create load balancer → Application Load Balancer

72. Name: propertyops-alb.
73. Scheme: Internet-facing.
74. IP type: IPv4.
75. VPC: propertyops-vpc.
76. Mappings: select both public subnets.
77. Security group: propertyops-alb-sg.
78. Listener for first smoke test: HTTP 80 → forward to propertyops-tg.

Checkpoint: The ALB becomes Active and has a public DNS name.

8. Create ECS Fargate

8.1 Task execution IAM role

Console path: IAM → Roles → Create role

79. Trusted entity: AWS service → Elastic Container Service → Elastic Container Service Task.
80. Attach AmazonECSTaskExecutionRolePolicy.
81. Name: propertyops-task-execution-role.
82. If the task definition reads Secrets Manager, add least-privilege permission for the propertyops/database secret.

Checkpoint: Role trust relationship uses ecs-tasks.amazonaws.com.

8.2 CloudWatch log group

Console path: CloudWatch → Logs → Log groups → Create log group

83. Name: /ecs/propertyops.
84. Choose a short retention period for the lab (for example 7 or 14 days).

Checkpoint: The log group is ready before task launch.

8.3 Create ECS cluster

Console path: ECS → Clusters → Create cluster

85. Cluster name: propertyops-cluster.
86. Use AWS Fargate / serverless infrastructure.

Checkpoint: Cluster status is Active.

8.4 Create task definition

Console path: ECS → Task definitions → Create new task definition

87. Launch type: AWS Fargate.
88. Family: propertyops-task.
89. CPU/memory: start small for the lab (for example 0.5 vCPU / 1 GB if supported for your workload).
90. Task execution role: propertyops-task-execution-role.
91. Container name: propertyops-app.
92. Image URI: your private ECR URI.
93. Container port: 80.
94. Log driver: awslogs → /ecs/propertyops.
95. Add DB values from Secrets Manager rather than plaintext environment variables.

Checkpoint: A new task definition revision is created.

8.5 Create ECS service

Console path: ECS → propertyops-cluster → Services → Create

96. Compute options: Launch type → FARGATE.
97. Application type: Service.

98. Task definition: propertyops-task (latest revision).
99. Service name: propertyops-service.
100. Desired tasks: 2.
101. Networking: propertyops-vpc.
102. Subnets: select only propertyops-app-2a and propertyops-app-2b.
103. Security group: propertyops-ecs-sg.
104. Public IP: Turned off.
105. Load balancing: Application Load Balancer → propertyops-alb.
106. Target group: propertyops-tg; container port 80.
107. Deployment circuit breaker / rollback: enable if available.

Checkpoint: Two tasks reach RUNNING and the target group reports both targets Healthy.

8.6 Smoke test

```
curl http://<ALB-DNS-NAME>/health
curl http://<ALB-DNS-NAME>/
```

Expected: The health request returns JSON similar to {"status":"ok","service":"propertyops"} and the home page displays PropertyOps.

9. Add uploads, monitoring and scaling

9.1 S3 uploads bucket

Console path: S3 → Create bucket

108. Bucket name: propertyops-uploads-<unique-suffix>.
109. Block all public access: On.
110. Encryption: On.
111. Enable versioning if you want document recovery.
112. Later, grant the ECS task role only the required GetObject/PutObject permissions for this bucket.

Checkpoint: Bucket is private and encrypted.

9.2 CloudWatch alarms

Console path: CloudWatch → Alarms → Create alarm

113. Create ECS service CPU utilization alarm.
114. Create ECS service memory utilization alarm.
115. Create ALB HTTPCode_Target_5XX_Count alarm.
116. Create HealthyHostCount / UnHealthyHostCount alarm for the target group.
117. Create RDS CPU/storage/connections alarms as appropriate.

Checkpoint: Your dashboard/alarms give you evidence for monitoring and incident-response sections of the portfolio.

9.3 SNS notifications

Console path: SNS → Topics → Create topic

118. Create propertyops-alerts.
119. Add your email subscription and confirm it.
120. Use the topic as an action for key CloudWatch alarms.

Checkpoint: Trigger a test alarm and confirm you receive the notification.

9.4 ECS Service Auto Scaling

Console path: ECS → Cluster → Service → Auto Scaling

121. Minimum tasks: 2.
122. Maximum tasks: 4 (lab-friendly).
123. Use target tracking, e.g. CPU around 60% as a starting point.
124. Save the scaling policy.

Checkpoint: The service can scale out above two tasks when the target metric is exceeded.

10. Final validation checklist

☐ ALB spans two public subnets in separate Availability Zones.

☐ ECS tasks have no public IPs and run in two private app subnets.

☐ RDS is not publicly accessible and uses private DB subnets.

☐ ECS inbound traffic comes only from the ALB security group.

☐ RDS inbound PostgreSQL traffic comes only from the ECS security group.

☐ ECR image pulls work without a NAT Gateway.

☐ ECR API, ECR DKR, Logs and Secrets Manager interface endpoints are Available with Private DNS enabled.

☐ S3 gateway endpoint is associated with the private app route table(s).

☐ CloudWatch receives ECS logs.

☐ Target group health checks are healthy.

☐ CloudWatch alarms and SNS notifications are configured.

☐ ECS Service Auto Scaling is configured.

☐ S3 uploads bucket is private and encrypted.

11. Portfolio evidence to capture

- Architecture diagram.
- VPC subnet view showing public/app/DB segmentation across two AZs.
- Security group rules showing SG-to-SG references.
- VPC endpoint list showing ECR, Logs, Secrets Manager and S3.
- ECR repository with the PropertyOps image.
- ECS service with two healthy Fargate tasks.
- ALB target group with healthy IP targets.
- Private RDS configuration showing Public access = No.
- CloudWatch dashboard and one test alarm.
- Autoscaling policy.
- Short incident exercise: deliberately deploy a broken image or health path, observe failure, then recover.

Interview story: Explain why you chose VPC endpoints instead of NAT Gateways: the workload primarily needs private access to AWS services, so PrivateLink/gateway endpoints reduce public egress exposure and can be more cost-efficient for this traffic pattern. Mention that interface endpoints still incur charges and NAT may be added later for genuine third-party internet dependencies.

12. Safe teardown order (important for cost control)

125. Delete the ECS service and wait until tasks stop.
126. Delete the ECS cluster after the service is gone.
127. Delete the ALB and target group.
128. Delete the RDS database (take a final snapshot only if you want to keep data).
129. Delete interface VPC endpoints (ECR API/DKR, Logs, Secrets Manager).
130. Delete the S3 gateway endpoint.
131. Delete ECR images/repository if you do not need them.
132. Empty and delete the uploads S3 bucket if you do not need its contents.
133. Delete security groups that are no longer referenced.
134. Delete route tables/Internet Gateway, then delete subnets and VPC.
135. Check Billing/Cost Explorer and Resource Explorer for anything left behind.

Do not delete your AWS account credentials: The teardown is for project resources only. Keep your IAM/Terraform learning setup unless you intentionally want to remove it.

13. What comes next

Once this ClickOps version is working, rebuild the same platform with Terraform modules. Then add GitHub Actions with OIDC, security scanning (Trivy/Checkov/TFLint), automated ECS deployments, load testing and documented incident recovery. The Terraform version should reproduce the ClickOps architecture without manual resource creation.

Reference notes

- AWS ECR documentation: Fargate 1.4+ with ECR interface endpoints also needs the S3 gateway endpoint for image layers.
- AWS ECS documentation: Fargate tasks using awsvpc and a load balancer use target groups with target type IP.
- AWS ECS documentation: private Fargate tasks using awslogs need a CloudWatch Logs interface endpoint when there is no internet/NAT path.